

Annexe technique

Plan de scalabilité

Stratégie de montée en charge — MVP Wallet vers production

Objet de ce document

Ce document définit la stratégie de scalabilité du wallet, de l'architecture MVP actuelle jusqu'à une infrastructure de niveau production. Il analyse les goulots d'étranglement existants, détaille les leviers d'optimisation par palier de charge, et fournit les estimations de coût associées.

Il complète le cahier des charges (section 6 — Architecture SI) et justifie les choix technologiques au regard de leur potentiel d'évolution.

1. Analyse de capacité — Architecture MVP

L'architecture MVP repose sur une instance unique de chaque composant (Next.js + PostgreSQL + Docker Compose). Cette configuration est suffisante pour la démonstration et les tests, mais présente des limites identifiées dès 200 utilisateurs simultanés.

Hypothèses de calcul

- 1 requête API moyenne = 15–50 ms de traitement côté Next.js
- 1 transaction locale = 2–3 requêtes SQL (lecture solde + écriture transaction + mise à jour solde)
- 1 transaction inter-wallets = 4–6 requêtes SQL + appel HTTP externe
- Pool Prisma par défaut = 10 connexions PostgreSQL simultanées

Composant	Capacité estimée MVP	Goulot d'étranglement	Limite critique
API Next.js (1 instance)	~50 req/s	CPU monoprocess	~200 utilisateurs simultanés
PostgreSQL (1 instance)	~100 connexions simultanées	Connection pool par défaut Prisma (10)	Saturation dès 50 req/s soutenues
Docker Compose (1 hôte)	Ressources du serveur unique	Pas de réplication	Zéro tolérance aux pannes
Prisma ORM (pool défaut)	10 connexions max	Non configurable sans env variable	Bottleneck BDD immédiat à l'échelle
Pas de cache	Chaque requête → BDD	Historique transactions : lecture intensive	Dégradation linéaire avec les utilisateurs
Pas de CDN	Frontend servi depuis l'origine	Latence pour utilisateurs distants	Acceptable en MVP, critique en production

Goulot critique identifié — connection pool Prisma

Le pool par défaut de Prisma (10 connexions) est le premier point de saturation. Avec 50 requêtes simultanées nécessitant chacune une connexion BDD, le système commence à mettre en file d'attente les requêtes dès le MVP. Ce point doit être corrigé en Sprint 1 via la variable `DATABASE_CONNECTION_LIMIT` ou l'ajout de PgBouncer.

2. Paliers de montée en charge

La scalabilité est organisée en paliers progressifs, chacun activé lorsque les métriques de monitoring dépassent les seuils définis. Cette approche évite la sur-ingénierie prématurée tout en garantissant une trajectoire claire.

Palier	Volume cible	Utilisateurs actifs	Infra requise	Actions clés
MVP (actuel)	< 100 req/s	< 500	1 serveur Docker Compose, 1 BDD	Périmètre actuel — aucune action
v1	100–500 req/s	500–5 000	2–3 instances API, load balancer, Redis, pool BDD 50	Redis, connection pooling, auto-scaling horizontal
v2	500–2 000 req/s	5 000–50 000	Kubernetes, réplica BDD lecture, CDN, Redis Cluster	Read replicas PostgreSQL, CDN Cloudflare, Kubernetes HPA
Production	> 2 000 req/s	> 50 000	Multi-région, BDD sharding, queue asynchrone	Architecture événementielle, file de messages, WAF, SIEM

Principe directeur

Chaque palier est une réponse à une métrique mesurée, pas une décision anticipée. On ne passe à v1 que quand la latence p95 dépasse 500 ms ou que le taux d'erreur 5xx dépasse 1% en charge réelle. L'architecture actuelle est conçue pour supporter ce passage sans refonte majeure.

3. Stratégie base de données (PostgreSQL)

PostgreSQL est un choix solide pour un système de paiement : transactions ACID, support natif du partitionnement, réplication streaming éprouvée. Les leviers de scalabilité sont nombreux et s'activent progressivement.

Levier	Palier d'activation	Détail d'implémentation
Connection pooling (PgBouncer)	v1 — priorité haute	PgBouncer en mode transaction pooling devant PostgreSQL. Pool de 50 connexions côté BDD, illimité côté API. Variable DATABASE_URL pointe vers PgBouncer, pas directement vers PostgreSQL.
Read replicas (PostgreSQL)	v2	1 replica en streaming replication. Lecture (GET /transactions, GET /wallets) dirigée vers le replica. Écriture (POST /transactions) vers le primary. Séparation dans Prisma via datasource secondaire.
Index stratégiques	MVP → v1	Index sur from_wallet_id, to_wallet_id, status, created_at dans Transactions. Index sur user_id dans Wallets. Réduisent les full table scans sur l'historique.
Partitionnement des tables	v2	Partitionnement de Transactions par plage de dates (RANGE on created_at). Réduit la taille des partitions actives et accélère les purges RGPD.
Archivage des données froides	v2	Transactions > 12 mois déplacées vers une table d'archive. Accès via API dédiée /api/transactions/archive. Réduit la taille des tables actives.

3.1 Configuration Prisma recommandée par palier

MVP — Configuration immédiate à appliquer

Dans le fichier .env, ajouter :

```
DATABASE_URL="postgresql://user:pass@host:5432/db?connection_limit=20&pool_timeout=10"
```

connection_limit=20 : augmente le pool Prisma de 10 à 20 connexions.

pool_timeout=10 : requêtes en attente de connexion abandonnées après 10s plutôt que bloquées indéfiniment.

v1 — Ajout de PgBouncer (connexion pooler externe)

PgBouncer s'intercale entre l'API et PostgreSQL. L'API peut ouvrir 200 connexions vers PgBouncer, qui en maintient seulement 20–30 réelles vers PostgreSQL. Mode transaction pooling recommandé pour des APIs stateless comme Next.js.

```
DATABASE_URL="postgresql://user:pass@pgbouncer:6432/db?pgbouncer=true"
```

3.2 Index critiques à créer dès le MVP

Ces index réduisent les full table scans sur la table Transactions, qui croît linéairement avec l'activité.

```
CREATE INDEX idx_tx_from_wallet ON Transactions(from_wallet_id);
CREATE INDEX idx_tx_to_wallet   ON Transactions(to_wallet_id);
CREATE INDEX idx_tx_status      ON Transactions(status);
CREATE INDEX idx_tx_created_at  ON Transactions(created_at DESC);
```

4. Stratégie cache — Redis

Redis résout trois problèmes distincts dans ce projet : la réduction de la pression sur PostgreSQL, le rate limiting distribué entre plusieurs instances API, et l'idempotence des transactions. Ces trois usages justifient à eux seuls son intégration dès la v1.

Usage Redis	Palier	Implémentation
Cache des wallets (solde)	v1	Cache du solde par wallet_id avec TTL 30s. Invalidation sur chaque POST /transactions. Réduit les lectures BDD de ~80% sur les endpoints dashboard.
Cache de l'historique transactions	v1	Cache paginé par wallet_id + page. TTL 60s. Clé : hist:{wallet_id}:{page}:{limit}.
Rate limiting distribué	v1 — priorité haute	Compteur par IP + endpoint avec sliding window (lua script Redis). Partage l'état entre plusieurs instances API — impossible avec un rate limiter en mémoire.
Sessions / Refresh tokens	v1	Stockage des refresh tokens avec TTL 7 jours. Permet la révocation immédiate sans rotation du JWT_SECRET.
Idempotence des transactions	v1	Clé idempotency_{transaction_id} avec TTL 24h. Si une requête POST /transactions arrive deux fois avec le même ID, la seconde est ignorée et renvoie le résultat de la première.
File de messages légère	v2	Redis Streams pour les événements inter-wallets asynchrones. Remplace l'appel synchrone Wallet A → Wallet B par un modèle publish/subscribe.

4.1 Stratégie d'invalidation du cache

Règle d'invalidation — Write-through

- À chaque POST /transactions (succès) : invalider le cache du solde des deux wallets concernés et la première page de l'historique de chacun.
- À chaque POST /inter-wallet/transfer (succès) : invalider le cache du wallet source uniquement (le wallet destination appartient à un autre système).
- Les clés de cache expirent via TTL même sans invalidation explicite — filet de sécurité garanti.
- Ne jamais mettre en cache les données financières critiques (solde) avec un TTL > 60s — risque d'afficher un solde périmé à l'utilisateur.

5. Infrastructure cloud par palier

L'architecture cloud retenue suit le principe de progression sans rupture : chaque palier réutilise les composants du précédent et en ajoute de nouveaux. Le passage MVP → v1 ne nécessite pas de réécriture — uniquement du déploiement et de la configuration.

Composant	MVP	v1	v2 / Production
API Next.js	1 conteneur Docker	2–3 instances ECS/Cloud Run + ALB	Kubernetes HPA (auto-scaling sur CPU/RPS)
Base de données	PostgreSQL RDS t3.micro	RDS t3.medium + PgBouncer	RDS Multi-AZ + 1 replica lecture + PgBouncer
Cache	Aucun	ElastiCache Redis t3.small	ElastiCache Redis Cluster (3 nœuds)
Load balancer	Aucun (1 instance)	AWS ALB ou GCP Load Balancer	ALB + WAF + CloudFlare
Frontend	Vercel / Netlify (auto-scale)	Vercel / Netlify + CDN intégré	Vercel Enterprise ou CloudFlare Pages
Stockage fichiers	Non applicable	S3 ou GCS pour exports RGPD	S3 + CloudFront pour assets statiques
Monitoring	Logs Docker	CloudWatch / GCP Monitoring + alertes	Datadog ou Grafana Cloud + PagerDuty
CI/CD	GitHub Actions (build + test)	GitHub Actions + déploiement ECS auto	GitHub Actions + blue/green deployment

5.1 Architecture cible v1 — Schéma logique

Flux d'une requête en v1

Utilisateur → CloudFlare (DNS + protection DDoS basique) → AWS ALB (load balancer) → 2-3 instances ECS/Fargate (API Next.js) → PgBouncer → PostgreSQL RDS

Lectures lourdes (historique) : instances API → Redis (cache hit) → retour immédiat sans toucher la BDD

Cache miss : instances API → PgBouncer → PostgreSQL → résultat stocké dans Redis → retour utilisateur

5.2 Auto-scaling horizontal (v1 → v2)

Politique d'auto-scaling recommandée

- Métrique déclencheur : CPU moyen > 70% sur 3 minutes OU latence p95 > 800 ms
 - Scale out : +1 instance toutes les 2 minutes jusqu'à max_instances = 10
 - Scale in : CPU < 30% sur 10 minutes consécutives → retrait d'une instance
 - Minimum d'instances : 2 en permanence (tolérance aux pannes d'une instance)
 - L'API Next.js est stateless par construction — aucun état local, scale out immédiat sans coordination
-

6. Monitoring & observabilité

La scalabilité sans observabilité est aveugle. Ce plan de monitoring définit les métriques clés, les seuils d'alerte et les actions associées pour chaque composant critique.

Métrique	Outil	Seuil d'alerte	Action déclenchée
Latence API p95 (ms)	CloudWatch / Datadog	> 500 ms	Scale out API + investigation
Taux d'erreur HTTP 5xx	CloudWatch	> 1% sur 5 min	Alerte PagerDuty + rollback
Connexions BDD actives	pg_stat_activity	> 80% du pool max	Scale PgBouncer + alerter DBA
Hit rate cache Redis	Redis INFO stats	< 70%	Réviser les TTL ou étendre les clés cachées
Transactions BLOCKED / heure	Logs applicatifs	> 50/h	Réviser les seuils du moteur de fraude
CPU API (moyen)	CloudWatch	> 70% sur 10 min	Auto-scaling horizontal (HPA Kubernetes)
Taille des tables BDD	pg_relation_size	Transactions > 10 Go	Déclencher partitionnement ou archivage
Délai de réplication BDD	pg_stat_replication	> 30 secondes	Alerte — risque de données périmées sur replica

6.1 Stack de monitoring recommandée par palier

Palier	Stack recommandée	Justification
MVP	Logs Docker + CloudWatch basique	Gratuit, suffisant pour détecter les pannes évidentes
v1	CloudWatch Metrics + Alertes SNS + dashboard Grafana	Visualisation des métriques en temps réel, alertes email/Slack
v2	Datadog ou New Relic — APM complet + tracing distribué	Traçage des requêtes de bout en bout (API → BDD → cache), détection des slow queries
Production	Datadog + PagerDuty + Sentry (erreurs front)	Astreinte on-call, escalade automatique, error tracking

6.2 SLO (Service Level Objectives) cibles

Objectifs de niveau de service à définir avant v1

- **Disponibilité** : 99,5% mensuel (≈ 3,6h d'indisponibilité autorisée/mois) — réaliste en v1 sans Multi-AZ complet

- **Latence p95** : < 500 ms sur POST /transactions, < 200 ms sur GET /wallets
 - **Latence p99** : < 2 000 ms (transactions inter-wallets avec appel HTTP externe)
 - **Taux d'erreur** : < 0,5% de requêtes 5xx sur une fenêtre glissante de 5 minutes
 - **RTO (Recovery Time Objective)** : < 15 minutes en cas de panne d'instance (redémarrage auto ECS)
 - **RPO (Recovery Point Objective)** : < 1 heure (backup PostgreSQL RDS automatique toutes les heures)
-

7. Estimation des coûts par palier

Estimations basées sur les tarifs AWS EU (eu-west-3, Paris) en mode on-demand. Les coûts réels peuvent être réduits de 30 à 50% avec des Reserved Instances (engagement 1 an) ou Savings Plans.

Ressource	MVP (€/mois)	v1 (€/mois)	v2 (€/mois)	Notes
API (instances)	30	120–200	400–800	ECS Fargate ou GKE Autopilot
PostgreSQL (RDS)	10	80–150	300–600	Multi-AZ = x2 en v2
Redis (ElastiCache)	0	30–60	120–250	Cluster en v2
Load balancer	0	20–30	40–60	ALB AWS
CDN / Frontend	0	0–20	20–80	Vercel inclus jusqu'à un certain trafic
Monitoring	0	30–60	100–300	Datadog ou CloudWatch
Stockage & logs	0	10–20	30–80	S3 + rétention 90j
TOTAL ESTIMÉ	≈ 55 €	≈ 290–540 €	≈ 1 010–2 170 €	Hors équipe, hors support

Optimisations de coût recommandées dès v1

- Utiliser AWS Graviton (instances ARM) pour l'API : -20% de coût à performance équivalente
- Reserved Instances RDS 1 an : -40% sur le coût PostgreSQL
- S3 Intelligent-Tiering pour les logs d'audit : passage automatique en stockage froid après 30 jours
- Vercel/Netlify pour le frontend : gratuit jusqu'à 100 Go de bande passante/mois — aucun coût CDN en v1

8. Roadmap de scalabilité

Action	Palier	Effort estimé	Impact attendu
Augmenter pool Prisma (connection_limit=20)	MVP — immédiat	< 1h (config)	Supprime le goulot BDD dès 50 req/s
Créer les 4 index BDD critiques	MVP — immédiat	< 1h (migration)	Réduit les slow queries sur historique
Rate limiting Redis en mémoire (single instance)	Sprint 1 restant	4–8h dev	Protège les endpoints sensibles dès le MVP
Déploiement multi-instances ECS + ALB	v1	1–2 jours DevOps	Tolérance aux pannes + scale horizontal
Ajout Redis (ElastiCache) — cache + rate limiting distribué	v1	2–3 jours dev	Réduction pression BDD de ~60%, rate limiting multi-instances
PgBouncer en transaction pooling	v1	1 jour DevOps	Supporte 10x plus de connexions API simultanées
Monitoring CloudWatch + alertes Slack	v1	1 jour DevOps	Visibilité opérationnelle, détection proactive
Read replicas PostgreSQL	v2	1 jour + migration Prisma	Décharge les lectures du primary de ~70%
Kubernetes HPA (auto-scaling)	v2	3–5 jours migration K8s	Scale automatique sans intervention humaine
Partitionnement tables Transactions	v2	2–3 jours DBA	Maintient les performances à très fort volume de données
Architecture événementielle (Redis Streams / Kafka)	Production	1–2 semaines	Découplage complet Wallet A / Wallet B — scalabilité indépendante